

Sistemas Distribuídos — Aula 21

Sistemas Tolerantes a Falhas I: Modelos de Falha e Detecção · 27/10 · Prof. Leandro Borges

Por que tolerar falhas é central em sistemas distribuídos

Nas últimas aulas, olhamos para como processos distribuídos coordenam decisões (consenso, na Aula 18), capturam um retrato consistente do sistema (snapshots e detecção de terminação, na Aula 19) e mantêm múltiplas cópias dos mesmos dados de acordo entre si (replicação e consistência, na Aula 20). Em todos esses casos, havia uma suposição implícita por trás do raciocínio: que processos e mensagens podem falhar a qualquer momento, e que o algoritmo precisa continuar funcionando mesmo assim. Esta aula torna essa suposição explícita e é a primeira de três dedicadas ao tema: hoje tratamos de como um sistema distribuído pode falhar e como detectamos que uma falha aconteceu; a Aula 22 usa redundância e replicação para mascarar falhas depois de detectadas; e a Aula 23 fecha o bloco com recuperação e um estudo de caso real.

A razão pela qual tolerância a falhas ocupa um lugar tão central em sistemas distribuídos é simples de enunciar e difícil de evitar: quanto mais componentes um sistema tem — processos, máquinas, discos, enlaces de rede —, maior é a probabilidade combinada de que algum deles esteja falhando a qualquer instante. Em um sistema centralizado, uma falha geralmente significa que o sistema inteiro para; o operador percebe imediatamente e reage. Em um sistema distribuído, o cenário mais comum é bem diferente: uma falha parcial, em que parte do sistema continua funcionando normalmente enquanto outra parte já não responde, ou responde de forma incorreta. É esse comportamento — funcionar parcialmente, de forma imprevisível, sem um sinal único e óbvio de que algo deu errado — que torna a tolerância a falhas um problema de projeto, e não um detalhe de implementação a ser tratado depois.

Tolerar falhas de forma sistemática exige duas coisas em sequência. Primeiro, um vocabulário preciso sobre os modos possíveis de falha: dizer apenas que 'algo pode falhar' não ajuda a projetar um algoritmo — é preciso saber se o processo parou de vez, se apenas perdeu uma mensagem, se está respondendo atrasado ou se está respondendo errado. Segundo, um mecanismo para descobrir, em tempo de execução, que uma falha de fato ocorreu, já que sem detecção nenhuma medida de correção pode ser acionada. Essas duas peças — modelos de falha e detecção de falha — são o assunto desta aula.

Modelos de falha: a taxonomia de Cristian

Uma forma amplamente usada de organizar os tipos de falha em sistemas distribuídos é a taxonomia proposta por Flaviu Cristian em 1991, adotada tanto por Tanenbaum e Van Steen quanto por Coulouris e colegas como ponto de partida para o assunto. Ela classifica as falhas pelo efeito observável que produzem, dos casos mais simples de tratar até os mais difíceis.

A falha de colapso, ou crash failure, é a mais simples: o processo simplesmente para de executar e não retoma — ele não emite mais nenhuma mensagem, não responde a mais nenhuma requisição, e esse silêncio é definitivo. A falha de omissão, ou omission failure, é mais sutil: o processo continua ativo, mas falha em enviar ou em receber uma mensagem específica que deveria ter sido trocada — fala-se em falha de omissão de envio quando uma mensagem que deveria sair da fila de saída se perde, e falha de omissão de recepção quando uma mensagem chega fisicamente mas o processo não a processa.

A falha de temporização, ou timing failure, ocorre quando a resposta do processo está correta em conteúdo, mas chega fora do intervalo de tempo esperado — cedo ou tarde demais —, um problema especialmente relevante em sistemas de tempo real ou de mídia contínua, em que um dado correto mas atrasado já não serve para nada. A falha de resposta, ou response failure, acontece quando o processo responde dentro do prazo, mas a resposta em si está incorreta: pode ser uma falha de valor, em que o conteúdo devolvido está errado (por exemplo, um saldo bancário incorreto), ou uma falha de transição de estado, em que o processo se desvia do fluxo de controle correto especificado pelo algoritmo.

No topo da escala de gravidade está a falha arbitrária, também chamada falha bizantina: o processo pode produzir qualquer resposta, em qualquer momento, sem nenhum padrão detectável — incluindo, no caso extremo, um comportamento deliberadamente malicioso, como um nó comprometido que forja dados para enganar o restante do sistema. Cada categoria dessa taxonomia é, em certo sentido, um caso particular da categoria seguinte: toda falha de colapso é também uma falha de omissão (o processo para de enviar tudo), e toda falha pode, no limite, ser modelada como arbitrária. Mas assumir o modelo mais fraco possível — geralmente colapso — sempre que a aplicação permite é o que torna o projeto de algoritmos tolerantes a falhas tratável na prática.

Fail-stop versus falhas silenciosas e bizantinas

Dentro da categoria de falhas de colapso existe uma distinção importante para quem projeta algoritmos distribuídos: a falha fail-stop. Um processo falha de maneira fail-stop quando, além de parar definitivamente, essa parada pode ser detectada com certeza pelos demais processos do sistema — por exemplo, porque o ambiente de execução garante um limite superior confiável para o atraso de mensagens, o que torna um timeout um sinal inequívoco de falha. Fail-stop é a suposição mais confortável sob a qual projetar um sistema tolerante a falhas: como existe um sinal limpo e detectável de que o processo morreu, os demais participantes podem reagir de forma determinística — removê-lo do grupo, redistribuir seu trabalho, eleger um substituto — sem ambiguidade.

O problema é que a maior parte dos sistemas distribuídos reais não oferece essa garantia. Em uma rede assíncrona, comum na internet e na maioria das redes corporativas, não existe um limite superior garantido para o atraso de uma mensagem: um processo que está apenas lento, sobrecarregado, ou momentaneamente isolado por congestionamento de rede, é observado exatamente da mesma forma que um processo que já falhou — o silêncio, sozinho, não diz qual dos dois cenários está ocorrendo. Esse é o terreno das falhas silenciosas: o processo pode estar vivo, mas inacessível por tempo suficiente para ser confundido com morto.

No extremo mais difícil de tratar estão as falhas bizantinas, nome que remete ao chamado Problema dos Generais Bizantinos, formulado por Lamport, Shostak e Pease em 1982 para descrever a dificuldade de obter acordo confiável entre partes que podem mentir. Uma falha bizantina não produz apenas silêncio ou lentidão: o processo continua respondendo, mas pode fazê-lo de forma incorreta, inconsistente para observadores diferentes, ou ativamente enganosa. Não há, nesse caso, nenhum sinal estrutural de falha para se reagir — o algoritmo precisa ser construído para tolerar que alguns participantes mintam, e não apenas que alguns participantes sumam. É por isso que sistemas que assumem apenas o modelo de colapso são, em geral, mais simples e mais baratos de construir do que sistemas que precisam resistir a comportamento bizantino, e por isso a maioria dos sistemas distribuídos de uso comum é projetada assumindo colapso (ou, na melhor das hipóteses, fail-stop), reservando tolerância bizantina para cenários de alta desconfiança, como blockchains e sistemas com participantes não confiáveis.

Mecanismos de detecção: heartbeat, ping e timeout

Independentemente do modelo de falha assumido, um sistema tolerante a falhas precisa de algum mecanismo concreto para descobrir que um processo parou de responder. Os dois mecanismos mais básicos e amplamente usados são simétricos entre si. No modelo de heartbeat, o próprio processo monitorado assume a iniciativa: ele envia periodicamente uma mensagem curta de 'estou vivo' para um monitor (ou para os demais membros do grupo). Enquanto os heartbeats continuam chegando dentro do intervalo esperado, o processo é considerado ativo; a ausência de heartbeat por um número de períodos consecutivos é interpretada como suspeita de falha.

No modelo de ping, ou 'are-you-alive', a iniciativa é do monitor: ele envia periodicamente uma requisição perguntando se o processo monitorado ainda está ativo, e aguarda uma resposta dentro de um prazo definido — o timeout. Se a resposta chega dentro do prazo, o processo é considerado vivo; se o prazo expira sem resposta, o processo é declarado suspeito de falha. Os dois modelos resolvem o mesmo problema com o fluxo de iniciativa invertido, e sistemas reais frequentemente combinam os dois — heartbeats para o caso comum, complementados por pings pontuais quando um heartbeat esperado não chega.

Em qualquer um dos dois modelos, o timeout é o parâmetro que efetivamente decide o que conta como falha. E é justamente aqui que a diferença entre um sistema síncrono e um assíncrono se torna prática: em um sistema síncrono, onde existe um limite superior conhecido para o atraso de mensagens e para o tempo de processamento, um timeout bem calibrado é um teste confiável — se o prazo expirou, o processo realmente falhou. Em um sistema assíncrono, esse limite superior não existe (ou não é conhecido com precisão), o que significa que nenhum valor de timeout garante, sozinho, a distinção perfeita entre 'processo morto' e 'processo apenas lento'.

Failure detectors não confiáveis e o trade-off da detecção

Essa limitação fundamental dos sistemas assíncronos foi formalizada por Tushar Chandra e Sam Toueg em um artigo de 1996 que se tornou referência no assunto, introduzindo a noção de detector de falha não confiável (unreliable failure detector). Um failure detector, nesse sentido, é um módulo associado a cada processo que mantém uma lista dos processos que ele atualmente suspeita estarem falhos, com base em observações locais — tipicamente, timeouts sobre heartbeats ou pings. A palavra 'não confiável' no nome é deliberada: o detector pode errar em ambas as direções, suspeitando de um processo que na verdade está ativo (falso positivo, geralmente causado por lentidão de rede) ou demorando a suspeitar de um processo que já falhou de fato.

Para que essa incerteza ainda seja útil na prática, Chandra e Toueg descrevem os failure detectors por duas propriedades desejáveis. A completude exige que, eventualmente, todo processo que de fato falhou passe a ser suscitado por todos os processos corretos — ou seja, uma falha real nunca fica permanentemente sem ser notada. A acurácia, em suas várias formas mais fracas ou mais fortes, limita o quanto o detector pode errar ao suspeitar de processos que continuam corretos. Diferentes classes de detectores (perfeito, eventualmente perfeito, entre outras) correspondem a diferentes combinações dessas garantias, e são a base teórica sobre a qual algoritmos de consenso tolerantes a falhas — como os vistos na Aula 18 — se apoiam para funcionar mesmo em ambientes assíncronos.

Na prática de engenharia, essa incerteza teórica se traduz em uma decisão de projeto muito concreta: o valor do timeout. Um timeout curto detecta falhas reais rapidamente, o que é valioso porque reduz o tempo em que o sistema continua operando sem saber que um componente está fora do ar — mas paga esse ganho com mais falsos positivos, já que qualquer lentidão momentânea de rede passa a ser confundida com uma falha real, disparando reações desnecessárias (como remover um processo saudável do grupo ou promover um substituto sem necessidade). Um timeout longo faz o caminho inverso: reduz falsos positivos, porque dá tempo suficiente para respostas legítimas

atrasadas chegarem, mas reage mais devagar a falhas reais, deixando o sistema operar por mais tempo sem se ajustar a uma falha que já aconteceu. Não existe um valor de timeout que elimine os dois problemas ao mesmo tempo em um sistema assíncrono; o que existe é um ponto de equilíbrio, escolhido de acordo com qual dos dois erros — reagir tarde demais ou reagir por engano — custa mais caro para aquela aplicação específica.

Próxima aula

Hoje ficamos deliberadamente no diagnóstico: como um sistema distribuído pode falhar e como percebemos que uma falha ocorreu. Saber diagnosticar uma falha, porém, não é o mesmo que sobreviver a ela — um failure detector que aponta corretamente um processo morto não é de nenhuma utilidade se o sistema não tiver como continuar funcionando sem ele. A Aula 22 dá esse próximo passo: replicação e redundância como mecanismos para mascarar falhas depois de detectadas, retomando o tema de réplicas de dados já visto na Aula 20, mas agora sob a ótica de tolerância a falhas — quantas cópias manter, onde posicioná-las e como reagir automaticamente quando uma delas some. A Aula 23 fecha o bloco de três aulas com estratégias de recuperação após uma falha e um estudo de caso real de sistema tolerante a falhas.

Referências

TANENBAUM, Andrew S.; VAN STEEN, Maarten. Distributed Systems: Principles and Paradigms (edição atual: Distributed Systems). Pearson. Capítulo sobre tolerância a falhas: modelos de falha, fail-stop e failure detectors.

COULOURIS, George et al. Sistemas Distribuídos: Conceitos e Projeto. 5ª ed. Porto Alegre: Bookman. Capítulo sobre tolerância a falhas: taxonomia de falhas de Cristian e mecanismos de detecção.

CHANDRA, Tushar Deepak; TOUEG, Sam. "Unreliable Failure Detectors for Reliable Distributed Systems". Journal of the ACM, v. 43, n. 2, 1996. Artigo original que formaliza os detectores de falha não confiáveis.

LAMPORT, Leslie; SHOSTAK, Robert; PEASE, Marshall. "The Byzantine Generals Problem". ACM Transactions on Programming Languages and Systems, v. 4, n. 3, 1982. Artigo que introduz o problema e o nome das falhas bizantinas.